Understanding XAI on the NSL-KDD Dataset

A Beginner's Guide for New AI/ML Researchers

JAC Hermocilla and JM Raqueno

May 5, 2026

Outline I

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports

Outline II

8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

What This Tutorial Covers

Goal: Build an AI that detects network attacks *and* explains its reasoning

Dataset: NSL-KDD — 125,973 labelled network connections

Model: Random Forest (the "black box'')

Focus today: Steps 0–3 — setup, data loading, preprocessing, training

Overall roadmap:

```
Raw Data → Clean & Prepare → Train Model → Explain Decisions
 (NSL-KDD)     (Steps 0-2)       (Step 3)         (Steps 4-14)
```

Why Network Intrusion Detection?

- Thousands of connections flow through a company network every second
- Most are **legitimate** — employees checking email, browsing, uploading files
- Occasionally, a **hacker** tries to break in

The challenge has two parts:

1. Detect which connections are attacks **accurately**
2. Explain **why** the system flagged each one

Explainable AI (XAI)

We want a system that detects cyberattacks AND explains its reasoning so that security experts can trust, verify, and improve it.

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

NSL-KDD: Origins and Purpose

Background

NSL-KDD was created at the **University of New Brunswick** to address known problems with the original KDD Cup 1999 dataset, which was generated from 1998 DARPA network traffic simulations.

It remains the most widely used benchmark dataset for **Network Intrusion Detection System (NIDS)** research.

Improvements over KDD Cup 1999

- Removed duplicate records that caused classifiers to be biased
- Balanced class proportions between train and test sets
- Added a `difficulty` rating (1–21) per record
- Reduced dataset size to make full experiments feasible

NSL-KDD: Dataset Statistics

Property	Value
Training records	125,973
Test records	22,544
Feature columns	41
Label column	1
Difficulty column	1 (dropped)
Attack categories	5
Specific attacks	39
Source file	`KDDTrain+.txt`
Test file	`KDDTest+.txt`

Class distribution (training)

Class	Records	%
Normal	67,343	53.5%
DoS	45,927	36.5%
Probe	11,656	9.2%
R2L	1,126	0.9%
U2R	52	0.04%

Class imbalance

U2R has only 52 training examples — the rarest and most dangerous attack type. This is a known challenge.

NSL-KDD: Feature Groups

The 41 features are organised into four groups based on what aspect of the connection they measure:

Group	Count	What it captures
Basic connection	9	Duration, protocol, bytes, connection status (`duration`, `src_bytes`, `flag`, ...)
Content / payload	13	Login behaviour, privilege escalation, sensitive file access (`logged_in`, `root_shell`, ...)
Traffic (2-sec window)	9	Flooding and scanning bursts (`count`, `serror_rate`, `same_srv_rate`, ...)
Destination host (100 conn.)	10	Slow and distributed attacks (`dst_host_count`, `dst_host_serror_rate`, ...)

Attack signatures

DoS: `serror_rate`↑, `same_srv_rate`↓, `count`↑ Probe: `dst_host_diff_srv_rate`↑ R2L: `num_failed_logins`↑
U2R: `root_shell`=1, `su_attempted`=1

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

What is a Dataset?

A **dataset** is data organised like a spreadsheet

Component	Description
Row	One observation (one network connection)
Column	One piece of information (a **feature**)
Label column	The answer we want to predict

NSL-KDD at a glance:

Rows (connections)	125,973
Feature columns	41
Attack categories	5

What is a Feature?

A **feature** is one measurable characteristic of a network connection

Feature	What It Measures	Example
`duration`	Connection length (seconds)	0, 5, 300
`protocol_type`	Communication protocol	tcp, udp, icmp
`src_bytes`	Bytes sent from source	0, 1032, 50000
`serror_rate`	Rate of SYN errors	0.0 – 1.0
`logged_in`	Was login successful?	0 or 1

Analogy

Features are like a patient's vital signs. The label is the doctor's diagnosis.

What is a Label?

The **label** is the answer the model learns to predict

Label	Attack Type	Real-World Example
`Normal`	No attack	Employee checking email
`DoS`	Denial of Service	Flooding a server to crash it
`Probe`	Reconnaissance scan	Searching for open doors
`R2L`	Remote-to-Local	Guessing passwords
`U2R`	User-to-Root	Gaining admin privileges

What is Machine Learning?

Teaching computers by **example** rather than explicit rules

Traditional programming *(a human writes the rules manually)*:

```
IF src_bytes > 100000 AND serror_rate > 0.8
    THEN "Attack"
```

Machine learning *(the computer learns the rules itself)*:

```
Show the computer 100,000 labelled examples
Let it figure out the patterns automatically
```

Why ML?

The computer discovers patterns a human might never notice, and adapts when attackers change their tactics.

What is a Model?

A **model** is the result of training — the learned patterns

Three phases:

1. **Training** — model learns from labelled examples
2. **Inference** — trained model predicts labels for new data
3. **Evaluation** — we measure how accurate the predictions are

Analogy

A model is like a student who studied thousands of past exam questions. Given a new question, they use their learned knowledge to answer it.

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

What is a Decision Tree?

A **Decision Tree** makes predictions through a series of yes/no questions

Three parts of every tree:

Component	Also Called	Role
First question	**Root Node**	Most informative split
Middle questions	**Internal Nodes**	Refine the prediction
Final answers	**Leaf Nodes**	The actual prediction

The number of levels from root to leaf = the **depth** of the tree

The Decision Tree We Will Trace

```
   [ROOT] serror_rate > 0.5?
        /          \
     YES           NO
      |             |
same_srv_rate<0.1? logged_in=1?
   /     \       /     \
 YES    NO    YES    NO
  |      |      |      |
 DoS  Probe  Normal  R2L
```

Three features used:

- `serror_rate` — SYN error proportion
- `same_srv_rate` — same-service proportion
- `logged_in` — login success

Tracing a DoS Attack

Input connection record:

Feature | Value | Meaning |

|:——- -|:——|:———| | `serror_rate` | **0.92** | 92% SYN errors | | `same_srv_rate` | **0.04** | Only 4% same service | | `logged_in` | 0 | No login |

Path through the tree:

1. `serror_rate > 0.5?` → **0.92** → **YES** → go left
2. `same_srv_rate < 0.1?` → **0.04** → **YES** → go left

Prediction: DoS Attack

High SYN errors + scattered services = classic SYN Flood signature

Tracing a Normal Connection

Input connection record:

Feature	Value	Meaning
`serror_rate`	**0.00**	No SYN errors
`same_srv_rate`	0.88	*(not checked)*
`logged_in`	**1**	Login successful

Path through the tree:

1. `serror_rate > 0.5?` → **0.00** → **NO** → go right
2. `logged_in = 1?` → **1** → **YES** → go left

Prediction: Normal

No SYN errors + successful login = legitimate user activity

How Does a Tree Learn Its Questions?

The algorithm tries **every feature** and **every threshold**, picking the split that creates the purest groups

Gini impurity measures how mixed a group is:

- **Bad split:** both buckets still contain mixed class labels
- **Good split:** one bucket mostly DoS, the other mostly Normal

Marble analogy

Sorting a pile of mixed-colour marbles into buckets. The best split puts mostly red in one bucket and mostly blue in another. The algorithm greedily picks the "purest" split at every step.

Strengths and Weaknesses of a Single Tree

Strengths

- Human-readable — can be drawn
- Fast to train and predict
- No feature scaling needed
- Handles text and numbers

Weaknesses

- **Overfitting** — memorises instead of learning
- Sensitive to small data changes
- Less accurate than ensembles
- Deep trees become unreadable

Key takeaway

The overfitting problem is why we use a **Random Forest** instead of a single decision tree.

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

What is a Random Forest?

A **Random Forest** = many decision trees voting together

The problem with a single tree:
A single doctor can be biased, have knowledge gaps, or make a mistake.

The solution — two deliberate sources of randomness:

1. **Bootstrap Sampling:** Each tree trains on a *different random sample* of the training data (drawn with replacement)
2. **Random Feature Selection:** At each split, only a *random subset* of features is considered as candidates

This ensures trees are **diverse** — their mistakes cancel out when voting

How the Final Prediction Is Made

```
New connection arrives
         |
         v
+--- 100 Decision Trees ---+
| Tree 1:   DoS            |
| Tree 2:   DoS            |
| Tree 3:   Normal          |
| Tree 4:   DoS            |
| ...  (96 more)  ...     |
+--------------------------+
         |
         v
 DoS:  87 votes <-- WIN
 Normal: 8 votes
 Probe:  4 votes
 R2L:   1 vote
         |
         v
 Final: DoS
```

Majority Voting

Each tree casts one vote. The class with the most votes wins.

Analogy

Consult 100 doctors who each studied slightly different cases. The majority diagnosis is almost always more reliable than any single opinion.

Why “Random’’ Forest?

Random = two sources of randomness

1. Bootstrap sampling
   (different rows per tree)
2. Random feature subsets
   (different splits per tree)

Forest = many trees together

In our experiments

`n_estimators=100`
100 trees — a reliable default for most problems

Key insight

Diversity of trees is the source of strength. Each tree makes different mistakes, so they cancel out in the vote.

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

Installing the Libraries

```
!pip install shap PyALE scikit-learn \
    pandas numpy matplotlib seaborn --quiet
```

Library	Role
`pandas`	Load and manipulate data tables
`numpy`	Fast numerical computation
`matplotlib`	Draw charts and graphs
`seaborn`	Beautiful statistical plots
`scikit-learn`	ML algorithms and tools
`shap`	XAI explainability values
`PyALE`	Accumulated Local Effects plots

Note

Run this cell **once only** — libraries stay installed on your system.

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

Importing the Tools

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, \
                            accuracy_score
import shap
shap.initjs()
```

Import Lines Explained

`import pandas as pd`
Nickname pd for convenience

`warnings.filterwarnings('ignore')`
Suppress non-error yellow messages

`from sklearn.ensemble import RandomForestClassifier`
`ensemble` = group of models working together

`LabelEncoder`
Converts text categories to integers

`train_test_split`
Splits data into train and test portions

`shap.initjs()`
Initialises SHAP JavaScript visualisations

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

Step 2a: Define Column Names

The raw file has **no header row**:

```
0,tcp,http,SF,215,45076,0,0,0,0,0,1,...,normal,21
```

We define names manually so pandas can label each column:

```
col_names = ["duration", "protocol_type", "service",
             "flag", "src_bytes", "dst_bytes", ...
             "label", "difficulty"]

df = pd.read_csv("KDDTrain+.txt", names=col_names)
df.drop("difficulty", axis=1, inplace=True)
```

Why drop `difficulty`?

This column rates how hard each row is to classify — it describes the task, not the traffic. Keeping it would give the model an unfair, unrealistic advantage.

Step 2b: Encode Categorical Features

ML models only understand numbers — three features are text:

Feature	Example Values	Unique Count
`protocol_type`	tcp, udp, icmp	3
`service`	http, ftp, smtp, ...	66
`flag`	SF, REJ, RSTO, ...	11

```
for col in ["protocol_type", "service", "flag"]:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le
```

Result: `"tcp"` → 1, `"udp"` → 2, `"icmp"` → 0, ...

Step 2c: Map Labels to Attack Categories

39 specific attack names → 5 meaningful categories:

Original labels	Category
neptune, smurf, back, pod, teardrop	**DoS**
ipsweep, nmap, portsweep, satan	**Probe**
guess_passwd, ftp_write, imap, . . .	**R2L**
buffer_overflow, loadmodule, perl	**U2R**
normal	**Normal**

```
df['attack_cat'] = df['label'].map(attack_map) \
                              .fillna('Other')
df['label_enc'] = LabelEncoder() \
                  .fit_transform(df['attack_cat'])
```

Step 2d: Train / Test Split

Golden rule: Never evaluate on data the model trained on

```
X = df.drop(["label", "attack_cat", "label_enc"], axis=1)
y = df["label_enc"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,     # 20% held out for testing
    random_state=42,   # reproducible split
    stratify=y         # preserve class proportions
)
```

X and y

X = features (inputs), shape `(125973, 41)`
y = labels (outputs), shape `(125973,)`
From maths: $y = f(X)$

The split

Training set: 100,778 rows (80%)
Test set: 25,195 rows (20%)

Why `random_state=42` and `stratify=y`?

`random_state=42`

Seeds the random number generator so the **same split** is produced.

The value 42 is arbitrary — a convention referencing *The Hitchhiker's Guide to the Galaxy*.

`stratify=y`

Guarantees each class appears in the test set in the **same proportion** as the full dataset.

Without this, rare classes like U2R (0.04% of data) might appear too few times to evaluate fairly.

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

Creating and Training the Model

```
rf_model = RandomForestClassifier(
    n_estimators=100,  # 100 decision trees
    random_state=42,   # reproducibility
    n_jobs=-1          # use all CPU cores
)
rf_model.fit(X_train, y_train)
```

What `.fit()` does:

1. Takes `X_train` (features) and `y_train` (labels)
2. Builds 100 decision trees on different random data samples
3. Stores the entire learned forest inside `rf_model`

Training time

Typically **10–60 seconds** on the NSL-KDD training set, depending on your computer.

Evaluating the Model

```
y_pred = rf_model.predict(X_test)
acc    = accuracy_score(y_test, y_pred)
print(classification_report(y_test, y_pred,
                            target_names=CLASS_NAMES))
```

Typical output:

```
              precision  recall  f1-score  support
         DoS     0.99    0.99      0.99    22928
      Normal     0.99    0.99      0.99    67343
       Probe     0.98    0.97      0.98     5093
         R2L     0.97    0.96      0.96     1560
         U2R     0.88    0.89      0.88       49
```

Overall accuracy: ~**99%**

Understanding the Classification Report

Metric	Formula	Plain English
Precision	TP / (TP + FP)	Of all DoS predictions, how many were correct?
Recall	TP / (TP + FN)	Of all actual DoS attacks, how many were caught?
F1-Score	$2 \times P \times R$ / (P + R)	Balanced average of precision and recall
Support	Count of true examples	How many test rows belong to this class?

Why does U2R score lowest?

Only **49 U2R examples** exist in the test set. Too few examples to learn from — a classic **class imbalance problem**. Despite being the most dangerous attack type, it is the rarest in the data.

The Black-Box Problem

The problem

A Random Forest with 100 trees and 41 features is highly accurate — but nearly impossible to interpret directly.

Input goes in, output comes out. The internal reasoning is **hidden**.

The solution: XAI

Apply Model-Agnostic XAI methods to recover human-understandable explanations — without sacrificing accuracy.

The accuracy–interpretability trade-off:

```
Accurate ←----------------→ Interpretable
Random Forest   Neural Net     Decision Tree   Linear Model
```

Outline

1. Introduction
2. The NSL-KDD Dataset
3. Key Concepts
4. Decision Trees
5. Random Forests
6. Step 0: Install Dependencies
7. Step 1: Imports
8. Step 2: Load and Preprocess NSL-KDD
9. Step 3: Train the Black-Box Model
10. Checkpoint

What Have We Built?

Step	Accomplishment
0	Installed 7 Python libraries
1	Imported all tools and configured plot styles
2a	Loaded 125,973 labelled network connections
2b	Encoded text features (protocol, service, flag) into numbers
2c	Mapped 39 attack names into 5 categories
2d	Split data: 80% training, 20% test
3	Trained a Random Forest with 100 decision trees
3	Achieved ∼99% accuracy on completely unseen test data

Next steps

Apply XAI methods! We'll look into SHAP next

Glossary (1 / 2)

Term	Definition
Dataset	Data in rows (observations) and columns (features)
Feature	One measurable property of an observation
Label	The target to predict (e.g. `"DoS"`, `"Normal"`)
Model	A learned mapping from features to predictions
Root Node	First question in a decision tree
Leaf Node	Terminal node where prediction is made
Gini Impurity	Measure of class mixing after a split
Overfitting	Memorising training data, failing to generalise

Glossary (2 / 2)

Term	Definition
Random Forest	Ensemble of decision trees voting on predictions
Bootstrap Sampling	Each tree trains on a different random data sample
Encoding	Converting text categories to numbers
Train/Test Split	Dividing data to evaluate on unseen examples
Stratification	Preserving class proportions across the split
Class Imbalance	Some classes have far fewer examples than others
Black-Box Model	Accurate but internally unreadable model
XAI	Methods to make black-box decisions understandable

Thank You

Local and Global SHAP XAI on NSL-KDD

JAC Hermocilla and JM Raqueno

May 5, 2026

Section 1

Local and Global SHAP XAI on NSL-KDD

Outline

Global SHAP — explains overall model behaviour across all data

1. Global Bar Plot — mean |SHAP| per feature across all classes
2. Beeswarm Plots — per-class distribution of SHAP values

Local SHAP — explains individual predictions

3. Waterfall Plots — step-by-step feature contributions per sample
4. Force Plots — compact single-line view of all contributions

Section 2

Mathematical Background

Cooperative Game Theory: Shapley Values

SHAP is grounded in **cooperative game theory**. Consider a model f and a set of features $\mathcal{F} = \{1, 2, \ldots, p\}$.

The setup:

- Features are treated as **players** in a coalition game
- Each coalition $S \subseteq \mathcal{F}$ produces a **value** $v(S)$
- $v(S)$ = the model's expected output using only the features in S, marginalising over the rest
- The goal: fairly distribute the total payout $v(\mathcal{F})$ among all players

Analogy

Imagine four colleagues collaborate on a project. Each subset of colleagues produces some output. How much did each individual truly contribute? Shapley values give the **fair share** for each player.

The Shapley Value Formula

The Shapley value for feature i is the **weighted average marginal contribution** of feature i across all possible coalitions:

$$\phi_i(f, \mathbf{x}) = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \left[v(S \cup \{i\}) - v(S) \right]$$

Symbol	Meaning
p	Total number of features
S	A coalition not containing feature i
$v(S)$	Model output using only features in S
$v(S \cup \{i\})$	Output when i is added to coalition S
$v(S \cup \{i\}) - v(S)$	Marginal contribution of adding i
$\frac{\lvert S\rvert!(p-\lvert S\rvert-1)!}{p!}$	Weight: how often this coalition arises

Computational cost

Exact Shapley values require summing over 2^p subsets. For $p = 41$ features this is $2^{41} \approx 2$ trillion coalitions — infeasible directly.

SHAP uses efficient approximations (TreeExplainer, KernelExplainer) to make this tractable.

The Value Function $v(S)$

The value function $v(S)$ is defined as the expected model output when only the features in S are known:

$$v(S) = \mathbb{E}\left[f(\mathbf{x}) \mid \mathbf{x}_S\right] = \int f(\mathbf{x}_S, \mathbf{x}_{\bar{S}})\, p(\mathbf{x}_{\bar{S}} \mid \mathbf{x}_S)\, d\mathbf{x}_{\bar{S}}$$

Notation:

- $\mathbf{x}_S$ — observed values of features in coalition S
- $\mathbf{x}_{\bar{S}}$ — unknown features (complement of S)
- $p(\mathbf{x}_{\bar{S}} \mid \mathbf{x}_S)$ — conditional distribution of missing features

In practice: the conditional distribution is approximated by marginalising over the training data — replacing missing features with their observed values from other samples.

Baseline $E[f(\mathbf{x})]$

When $S = \emptyset$ (no features observed), $v(\emptyset) = \mathbb{E}[f(\mathbf{x})]$ — the model's average prediction across all training samples.

This is the **baseline** shown in waterfall and force plots as $E[f(x)]$.

The Four Shapley Axioms

Shapley values are the **unique** attribution that satisfies all four fairness axioms simultaneously:

Axiom	Statement	Plain English
Efficiency	$\sum_{i=1}^{p} \phi_i = f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})]$	SHAP values sum exactly to the prediction minus the baseline. Nothing is lost or double-counted.
Symmetry	If $v(S \cup \{i\}) = v(S \cup \{j\})\ \forall S$	Two features contributing equally to all coalitions receive equal SHAP values.
Dummy	If $v(S \cup \{i\}) = v(S)\ \forall S$	A feature that never changes any coalition's output gets $\phi_i = 0$.
Additivity	$\phi_i(f + g) = \phi_i(f) + \phi_i(g)$	SHAP values for a sum of models equal the sum of their individual SHAP values.

From Shapley Values to SHAP

SHAP (SHapley Additive exPlanations) frames the explanation as an **additive feature attribution model**:

$$f(\mathbf{x}) \approx g(\mathbf{x}') = \phi_0 + \sum_{i=1}^{p} \phi_i x_i'$$

Term	Meaning
g	The explanation model
$\mathbf{x}'$	Simplified binary input ($x_i' \in \{0, 1\}$)
$x_i' = 1$	Feature i is present (observed)
$x_i' = 0$	Feature i is absent (marginalised)
ϕ_0	Baseline $= \mathbb{E}[f(\mathbf{x})]$
ϕ_i	Shapley value for feature i

Key result (Lundberg & Lee, 2017)

Shapley values are the **only** solution satisfying local accuracy, missingness, and consistency simultaneously.

$\phi_i > 0$ — feature i pushes prediction **above** baseline
$\phi_i < 0$ — feature i pushes prediction **below** baseline
$\phi_i = 0$ — feature i has no effect

TreeExplainer: Efficient SHAP for Tree Models

For tree-based models (Random Forest, XGBoost), the exact Shapley values can be computed efficiently using the **TreeExplainer** algorithm (Lundberg et al., 2020).

Key idea:
Instead of summing over 2^p coalitions, TreeExplainer exploits the **tree structure** to compute exact Shapley values by propagating statistics through the tree nodes.

Complexity:

- Exact: $O(TLD2^M)$ where
- T = number of trees (100 here)
- L = max leaf count
- D = tree depth
- M = max features in any tree

In practice

For a Random Forest with 100 trees and depth 10–15, TreeExplainer computes exact SHAP values orders of magnitude faster than KernelExplainer (the model-agnostic approximation).

Code

```
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X)
```

Returns shape `[n_classes, n_samples, n_features]`

Global vs Local SHAP: Mathematical Link

Local and global SHAP are mathematically connected — global is simply an aggregation of local:

Local SHAP (per sample $\mathbf{x}^{(k)}$):

$$\phi_i^{(k)} \in \mathbb{R}$$

One value per feature per sample. Signed: direction matters.

Global SHAP importance (feature i):

$$\Phi_i = \frac{1}{n} \sum_{k=1}^{n} \left| \phi_i^{(k)} \right|$$

Mean absolute SHAP across all n samples.

Why absolute value?

$\phi_i^{(k)}$ can be positive or negative depending on the sample. Averaging signed values would cancel out. Taking $|\phi_i^{(k)}|$ first measures the **magnitude** of influence regardless of direction.

Consistency

The global bar plot IS the average of the absolute local waterfall bar heights. The two levels of explanation are fully consistent by construction.

Section 3

Global SHAP

Section 4

1. Global Bar Plot

Global Bar Plot — Discussion

- **Method:** SHAP uses game theory to fairly attribute model predictions to individual features. Aggregating absolute SHAP values across all samples gives a global importance ranking.
- **Interpretation:** A taller bar means the model relies more heavily on that feature overall. Bars are coloured by class contribution.

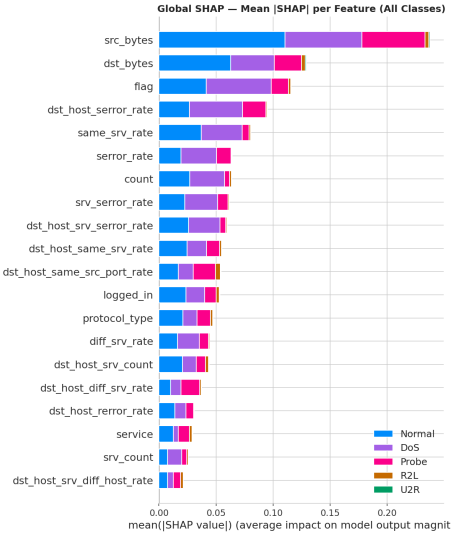
Top features found:

- `src_bytes` — most influential globally
- `dst_bytes`
- `serror_rate`
- `dst_host_same_src_port_rate`
- `count`

Key insight

Features dominating this plot are important across multiple attack classes. Features with low mean |SHAP| may still be critical for specific attack types — the beeswarm plots reveal this.

Global Bar Plot — Figure



Section 5

2. Beeswarm Plots

Beeswarm Plots — Discussion

- **Method:** Aggregates local SHAP values across all samples. One panel per attack class, showing the full distribution of feature contributions.
- **Interpretation:** Each dot is one sample.
  - **X position:** SHAP value — positive pushes prediction towards this class; negative pushes away
  - **Colour:** Feature value (red = high, blue = low)
  - **Y axis:** Features ranked by mean |SHAP| for this class

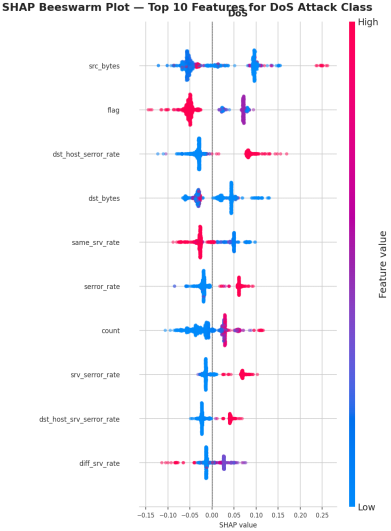
Example: DoS class

`serror_rate`: red dots far right → high SYN error rate strongly increases DoS probability.
`same_srv_rate`: blue dots far right → low same-service rate also increases DoS probability.

Expected attack signatures

DoS: `serror_rate`↑, `same_srv_rate`↓ Probe: `dst_host_diff_srv_rate`↑ R2L: `num_failed_logins`↑ U2R: `su_attempted`, `root_shell`

Beeswarm Plots — Figure



Section 6

Local SHAP

Section 7

3. Waterfall Plots

Waterfall Plots — Discussion

- **Method:** Decomposes one individual prediction into per-feature contributions, tracing the path from the model's baseline to its final output.
- **Interpretation:**
  - **Red bars** — feature pushes prediction higher (towards this class)
  - **Blue bars** — feature pushes prediction lower (away from this class)
  - $E[f(x)]$ — baseline: model's average output across all training data
  - $f(x)$ — final prediction for this specific connection
  - Features ordered by absolute SHAP magnitude; small contributions collapsed at bottom

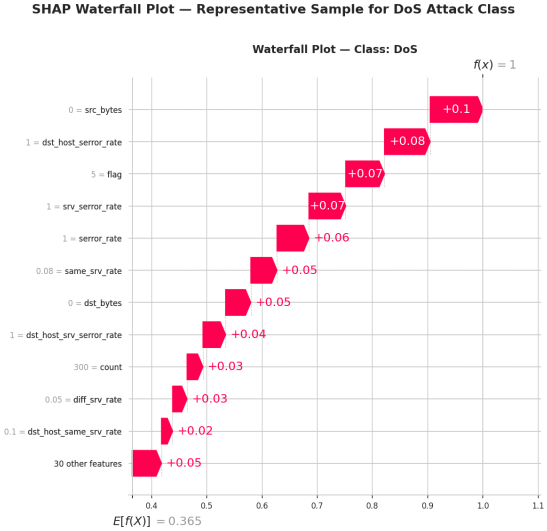
Example: DoS sample

Starting from $E[f(x)]$, a high `serror_rate` (red bar) pushes strongly towards DoS. A low `same_srv_rate` adds further evidence. The final $f(x)$ reflects all accumulated contributions.

Local vs Global

The waterfall may show a different top feature than the global bar plot. The global plot averages across all samples; the waterfall reflects this specific connection only.

Waterfall Plots — Figure



Section 8

4. Force Plots

Force Plots — Discussion

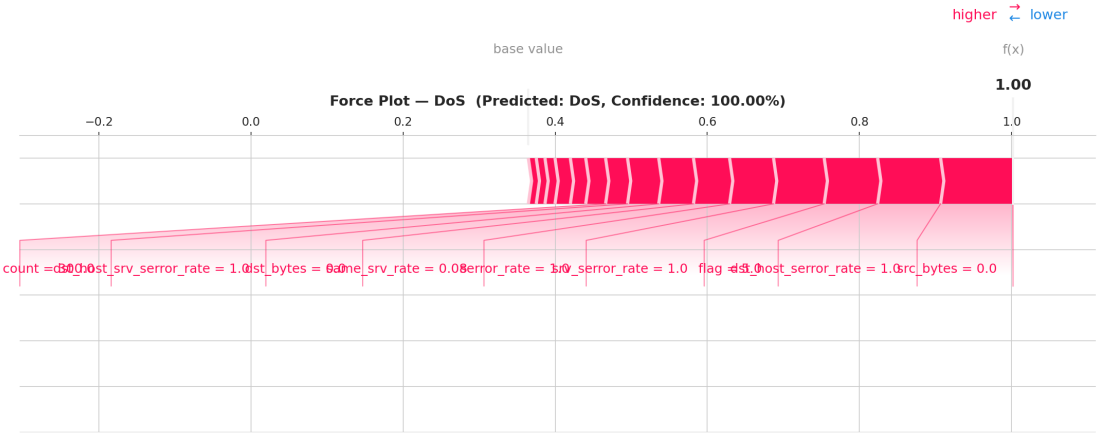
- **Method:** Presents the same information as the waterfall in a compact horizontal format. All feature contributions shown on a single axis simultaneously.
- **Interpretation:**
  - **Red region** — features pushing the prediction higher (towards this class)
  - **Blue region** — features pushing the prediction lower (away from this class)
  - The **width** of each segment reflects the magnitude of its contribution
  - Base value on the left; model output on the right

Property	Waterfall	Force Plot
Layout	Vertical stacked bars	Horizontal single axis
Best for	Detailed per-feature analysis	Quick push/pull summary

When to use

Use **waterfall** to explain a prediction in detail to a stakeholder. Use **force plot** for a quick summary or when comparing many predictions.

Force Plot — DoS Class



Section 9

Summary

Summary — Global vs Local SHAP

Property	Global SHAP	Local SHAP
Scope	All samples in dataset	One specific sample
Question	Which features matter overall?	Why this specific prediction?
Output	Ranked feature importance	Per-feature contribution values
Aggregation	Mean |SHAP| across samples	Individual SHAP values
Direction	Shown via beeswarm colour	Shown via bar colour (red/blue)

Key property of SHAP

Global and local explanations are fully consistent: the global bar plot is simply the average of the absolute local SHAP values. There is no disconnect between the two levels of explanation.